Jenkins CI/CD – Complete Practical Report

All requested topics, setup commands, Git commands, Jenkinsfile, trigger configuration and verification steps

Practical topic	Included content
Automate application deployment	Jenkins job, Poll SCM trigger, pipeline execution
Implement a basic Jenkins CI/CD pipeline	Build, Test and Deploy stages
Create a Jenkinsfile defining pipeline stages	Complete Declarative Jenkinsfile
Prepare a CI/CD workflow	GitHub → Jenkins → Build → Test → Deploy
Record all commands	Ubuntu/WSL setup, Git, GitHub CLI, verification and cleanup

1. Environment and tools

The practical was performed using Ubuntu on WSL2, Git, GitHub, and Jenkins. Jenkins itself ran on Windows, as indicated by the Jenkins workspace path C:\\ProgramData\\Jenkins\\.jenkins\\workspace\\GitHub-CI-CD. Run Linux commands in the Ubuntu/WSL terminal, not in Windows Command Prompt.

2. Install and verify Git, Java and Jenkins

Update Ubuntu packages and install Git, Java and required utilities:

```
sudo apt update
sudo apt upgrade -y
sudo apt install git -y
sudo apt install openjdk-21-jdk -y
sudo apt install wget -y

git --version
java -version
```

Configure Git identity:

```
git config --global user.name "Ar-Sawan"
git config --global user.email "iit.aspirant10@gmail.com"
git config --global --list
```

Jenkins package setup commands:

```
sudo mkdir -p /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/a

sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl status jenkins
```

Note: In this practical Jenkins was accessed through Windows at http://localhost:8080. The systemctl commands above are for a Linux-hosted Jenkins service; use Windows Services/Jenkins controls if Jenkins is installed natively on Windows.

3. Set up version control using Git

Create and initialize the local repository:

```
mkdir -p /mnt/c/Users/iitas/git-task
cd /mnt/c/Users/iitas/git-task
git init

echo "# Git Task Project" > README.md
```

```
git add README.md
git commit -m "Initial commit
```

Create a feature branch, commit a change, and merge it into master:

```
git branch feature
git switch feature

echo "This project demonstrates Git version control." >> README.md
git add README.md
git commit -m "Add project description"

git switch master
git merge feature
git branch -M master
git log --oneline --all --graph
```

Connect GitHub and push:

```
git remote add origin https://github.com/Ar-Sawan/git-task.git
git remote -v
git push -u origin master
```

GitHub CLI HTTPS authentication commands, if required:

```
sudo apt install gh -y
gh auth login
gh auth status
gh auth setup-git
git push -u origin master
```

If origin is already configured, do not add it again. Check with git remote -v first.

4. Create the Jenkinsfile

In Ubuntu/WSL, move into the project folder:

```
cd /mnt/c/Users/iitas/git-task
```

Create or replace Jenkinsfile with this Declarative Pipeline:

```
cat > Jenkinsfile <<'EOF'
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Build successful'
            }
        }

        stage('Test') {
            steps {
                echo 'Tests passed'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploy successful'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully!'
        }
    }
}
EOF
```

Verify, commit and push the Jenkinsfile:

```
cat Jenkinsfile
git status
git add Jenkinsfile
git commit -m "Add Jenkins declarative pipeline"
git push
```

Important: The Jenkinsfile must contain a stages { ... } block around the stage(...) entries. Do not put the shell wrapper text (cat > Jenkinsfile or EOF) inside the Jenkinsfile itself.

5. Configure the Jenkins pipeline job

In Jenkins, open the GitHub-CI-CD job and select Configure. Under Pipeline, use these values:

Setting	Value
Definition	Pipeline script from SCM
SCM	Git
Repository URL	https://github.com/Ar-Sawan/git-task.git
Branch Specifier	*/master
Script Path	Jenkinsfile

Click Save, return to the job page and select Build Now. Open the latest build and select Console Output.

6. Automate application deployment using Poll SCM

In the Jenkins job configuration, open Build Triggers and enable Poll SCM. Enter this schedule:

```
H/5 * * * *
```

This schedule asks Jenkins to poll the configured source-control repository approximately every five minutes. Poll SCM checks for changes; it does not itself guarantee production deployment. The pipeline runs when Jenkins detects a qualifying change.

7. Prepare the CI/CD workflow

Step	Activity
1. Developer	Updates files in the local Git repository.
2. Git	Stages and commits the changes.
3. GitHub	Receives the commit on the master branch.
4. Jenkins SCM	Checks out the repository and reads Jenkinsfile.
5. Build	Runs the Build stage.
6. Test	Runs the Test stage.
7. Deploy	Runs the Deploy stage.
8. Result	Jenkins reports success or failure in Console Output.

8. Verification commands

```
git status
git branch -vv
git log --oneline --all --graph
git remote -v
git push
cat Jenkinsfile
```

In Jenkins, verify that the latest build is successful and that Console Output includes Build, Test and Deploy stage messages followed by Finished: SUCCESS.

9. Expected Jenkins console output

```
Obtained Jenkinsfile from git https://github.com/Ar-Sawan/git-task

[Pipeline] { (Build)
Build successful

[Pipeline] { (Test)
Tests passed

[Pipeline] { (Deploy)
Deploy successful

Pipeline completed successfully!
Finished: SUCCESS
```

10. Practical limitation

The Jenkinsfile used in this practical prints success messages for Build, Test and Deploy. It demonstrates Declarative Pipeline structure and stage orchestration, but it does not compile a real application, run a test suite, or deploy an application to a live server. For a real deployment, replace the echo steps with the project's actual build, test and deployment commands.

11. Submission checklist

• Git version and global Git configuration screenshot.

• Repository, branches, commits and GitHub push screenshot.

• Jenkins Pipeline-from-SCM configuration screenshot.

• Poll SCM trigger and schedule screenshot, if required.

• Console Output showing Build, Test, Deploy and Finished: SUCCESS.

• Jenkinsfile saved in the GitHub repository.